

**Aufbau einer Datenabfrage in SQL**

Eine der wichtigsten Funktionen von Datenbanken ist das Auslesen von Daten. In SQL besteht der Befehl zur Abfrage von Datensätzen aus drei Bestandteilen:

- **SELECT:** zuerst werden die Tabellenspalten angegeben, die ausgelesen werden sollen. Ein * wählt dabei alle Spalten aus.
- **FROM:** es muss ausgewählt werden, aus welcher Tabelle die Daten stammen.
- **WHERE:** zuletzt können Einschränkungen gemacht werden, so dass nur bestimmte Datensätze ausgegeben werden.

Eine typische Abfrage sähe beispielsweise folgendermaßen aus:

```
1 | SELECT Name FROM Kunde WHERE GebDatum < '1980-01-01';
```

Der Befehl gibt die Namen aller Kunden aus, welche vor dem 01.01.1980 geboren sind.

**Vergleichs- und Logikoperatoren in SQL**

In WHERE-Einschränkungen können die Vergleichsoperatoren =, <, >, <=, >= und <> verwendet werden. Wesentliche Unterschiede im Vergleich zu typischen Programmiersprachen wie Python sind der Vergleich mit = (statt ==) und der Vergleich auf Ungleichheit mit <> (statt !=).

Zusätzlich können mehrere Bedingungen mit den Keywords AND, OR und NOT (sowie angemessener Klammerung) verknüpft werden.

**Komplexere Vergleiche**

Die Vergleiche können noch komplexer gestaltet werden, indem man den LIKE-Operator verwendet. Das Zeichen % steht dabei für eine beliebige Zeichenkette, während das Zeichen _ einen einzelnen Charakter darstellt.

Außerdem kann man mit BETWEEN überprüfen, ob ein Wert in einem bestimmten Bereich liegt. Der Operator IN überprüft, ob der gesuchte Wert in einer Liste ist. Die Liste wird mit kommagetrennten Werten in runden Klammern angegeben.

Komplexere Vergleiche könnten beispielsweise so aussehen:

```
1 | SELECT GebDatum FROM Kunde WHERE Name LIKE 'Max%';  
2 | SELECT * FROM Bestellung WHERE Preis BETWEEN 100 AND 1000;  
3 | SELECT Adresse FROM Kunde WHERE Name IN ('Max', 'Mia', 'Moritz')
```

Der erste Befehl gibt die Geburtstage aller Kunden aus, deren Name mit 'Max' beginnt. Der zweite Befehl gibt alle Bestellungen aus, deren Preis zwischen 100 und 1000 liegt. Der dritte Befehl gibt die Adressen aller Kunden aus, die 'Max', 'Mia' oder 'Moritz' heißen.



Bundesliga-Datenbank

Im Schulnetzwerk steht die PostgreSQL-Datenbank `bundesliga` zur Verfügung, die Daten über die vergangene Saison beinhaltet.

Mit dem folgenden Befehl und dem Passwort `postgres` kann man sich im Schulnetzwerk auf die Datenbank verbinden:

```
psql -h localhost bundesliga postgres
```

Die Bundesliga-Datenbank beinhaltet folgende Tabellen:

```
verein(vid, name, liga)  
spiel(nr, spieltag, datum, anstoss, heim, gast, tore_heim, tore_gast,  
      zuschauer, schiedsrichter)  
spieler(sid, verein, nachname, vorname, geburtsdatum, land, position,  
        spiele, tore)
```

Die unterstrichenen Attribute sind Primärschlüssel, gestrichelt unterstrichene Attribute sind Fremdschlüssel.



Aufgabe 1: Abfragen auf der Bundesliga-Datenbank

Verbinden Sie sich auf die Bundesliga-Datenbank und finden Sie die folgenden Daten. Notieren Sie jeweils den dazu verwendeten SQL-Befehl.

- Geben Sie alle Spieler aus.
- Geben Sie die Namen aller Vereine aus.
- Geben Sie die Namen aller Spieler aus, die "Thomas" heißen.
- Geben Sie die Spieltage aller Spiele aus, die mehr als 30000 Zuschauer hatten und bei denen die Gastmannschaft genau ein Tor geschossen hat.
- Finden Sie zuerst die Vereinsnummer des SC Freiburg heraus. Geben Sie dann die Namen aller Spieler aus, die beim SC Freiburg spielen.
- Geben Sie alle Spiele aus, die nach 19:00 Uhr begonnen haben.
- Geben Sie die Vereine, Vor- und Nachnamen aller Spieler aus, die über 10 Tore geschossen haben und älter als 20 Jahre sind.

- h) Geben Sie alle Spiele aus, bei denen die Heimmannschaft gewonnen hat.
- i) Geben Sie alle Vereine aus, deren Name "Berlin" beinhaltet.
- j) Geben Sie alle Spiele aus, die am 1., 17. oder 33. Spieltag stattgefunden haben.
- k) Geben Sie alle Spiele aus, die zwischen dem 18. und dem 34. Spieltag stattgefunden haben.
- l) Geben Sie alle Spiele aus, die im Februar stattgefunden haben.

Aufgabe 2: Eigene Abfragen formulieren

- a) Experimentieren Sie mit verschiedenen Abfragen herum und finden Sie interessante Befehle.
- b) Schreiben Sie selbst Aufgaben im Stil von Aufgabe 1.
- c) Tauschen Sie Ihre selbst geschriebenen Aufgaben mit Ihrem Partner und finden Sie gegenseitig die korrekten SQL-Befehle.
- d) Schreiben Sie weitere SQL-Befehle und tauschen Sie sie mit Ihrem Partner. Nun sollen Sie zu den vorgegebenen Befehlen angeben, welche Daten diese heraussuchen.